

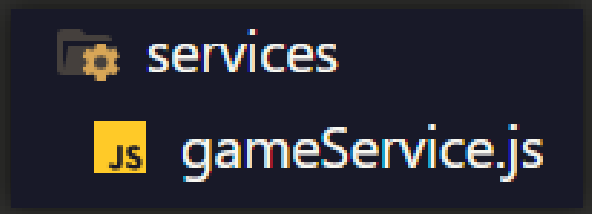


Aula 02:

Desenvolvendo o CRUD da API

Buscando registros na API

Após criarmos a base de dados da API, implementaremos agora um novo service na estrutura do projeto. Para isso, crie uma pasta na raiz do projeto chamada **services,** dentro desta pasta crie um novo arquivo chamado **gameService.js**.



No arquivo **gameService.js** criaremos uma **classe** que será responsável por conter os **métodos** de manipulação do banco de dados, como buscar, criar, alterar e deletar registros.

```
1 import Game from "../models/Games.js";
2
3 class gameService {
4   async getAll() {
5     try {
6       const games = await Game.find();
7       return games;
8     } catch (error) {
9       console.log(error);
10    }
11  }
12}
13export default new gameService();
```

O primeiro método a ser criado é o **getAll()**, que irá selecionar todos os registros de games cadastrados no banco de dados. Esse método será uma **função assíncrona**, por isso utilizaremos o padrão **async/await**.

Funções assíncronas (async/await)

O padrão **async/await** no JavaScript é uma maneira moderna e mais fácil de lidar com operações **assíncronas**, como buscar dados de uma API, sem precisar usar muitos callbacks ou **promises**.

Função assíncrona (async): Quando você define uma função como async, ela automaticamente retorna uma promessa e permite que você use o **await** dentro dela.

await: O await é usado dentro de funções async para "**esperar**" que uma promessa seja resolvida. Isso significa que o código vai parar e esperar até que a operação assíncrona termine, mas sem bloquear o resto do programa. Depois que a promessa é resolvida, o código continua a ser executado.

```
import Game from "../models/Games.js";

class gameService {
  async getAll() {
    try {
      const games = await Game.find();
      return games;
    } catch (error) {
      console.log(error);
    }
  }
}
export default new gameService();
```

Funções assíncronas (async/await)

Promisse:

```
import Game from "../models/Games.js";

class gameService {
  getAll() {
    return Game.find()
      .then(games => {
        return games;
      })
      .catch(error => {
        console.log(error);
      });
  }
}

export default new gameService();
```

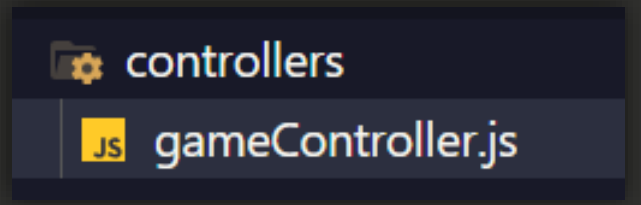
Async/await:

```
import Game from "../models/Games.js";

class gameService {
  async getAll() {
    try {
      const games = await Game.find();
      return games;
    } catch (error) {
      console.log(error);
    }
  }
}
export default new gameService();
```

Criando o Controller

Após criarmos o service, criaremos agora nosso **controller**. Para isso, crie uma pasta na raiz do projeto chamada **controllers,** dentro desta pasta crie um novo arquivo chamado **gameController.js**.

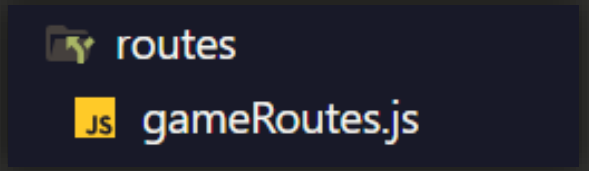



```
1 import gameService from "../services/gameService.js";
2
3 const getAllGames = async (req, res) => {
4   try {
5     const games = await gameService.getAll();
6     res.status(200).json({ games: games });
7   } catch (error) {
8     console.log(error);
9     res.status(500).json({ error: "Erro interno do servidor." });
10  }
11};
12export default { getAllGames };
```

O controller será responsável por **tratar as requisições do cliente**. No exemplo ao lado, **getAllGames** recebe uma *função assíncrona* que chama o método **getAll()** do **service** para buscar os registros no banco. Em seguida, o controller retorna esses registro em um **JSON**, bem como o **código de status 200**. Além disso, também é feito o tratamento de erro.

Modularizando as rotas

Visando a eficiência do projeto iremos agora modularizar as rotas da API (endpoints). Para isso, crie uma pasta na raiz do projeto chamada **routes,** dentro desta pasta crie um novo arquivo chamado **gameRoutes.js**.



Nesse arquivo iremos importar o express, em seguida carregar na variável **gameRoutes** o método **express.Router()**, responsável por fazer o gerenciamento de rotas. Importaremos também o **gameController** responsável por tratar as requisições.

```
import express from 'express'
const gameRoutes = express.Router()
import gameController from '../controllers/gameController.js'

// Endpoint para listar todos os games
gameRoutes.get("/games", gameController.getAllGames)

export default gameRoutes
```

Nesse arquivo criaremos todos os **endpoints** (rotas) da API. Começaremos criando a rota "**/games**" que ao receber um requisição do tipo **GET**, chamará o método **getAllGames()** do **gameController** para fazer o tratamento da requisição.

Listando os registros (GET)

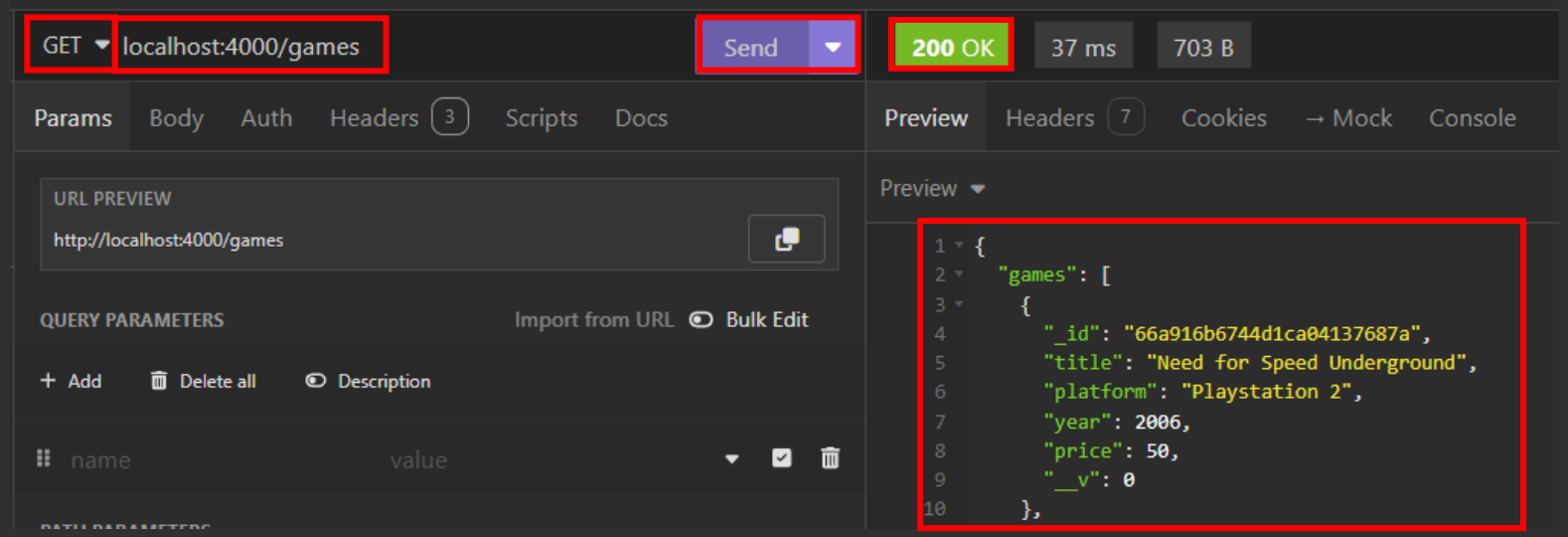
Por fim, devemos importar nossas rotas no arquivo **index.js**, para que sejam acessíveis na aplicação.

```
const app = express();

import gameRoutes from "./routes/gameRoutes.js";

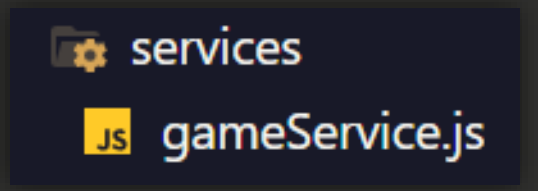
// Configurações do Express
app.use(express.urlencoded({ extended: false }));
app.use(express.json());
app.use("/", gameRoutes);
```

Feito isso, testaremos a requisição no **Insomnia**, enviando uma requisição **GET** para a rota "**/games**":



Cadastrando dados na API

Para começarmos a cadastrar dados na API, criaremos um novo método chamado **Create()** na classe **gameService** que irá inserir novos registros no banco de dados.

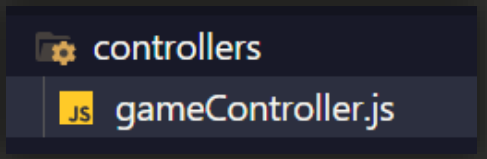



```
async Create(title, platform, year, price) {
    try{
        const newGame = new Game({
            title,
            platform,
            year,
            price
        })
        await newGame.save()
    } catch (error) {
        console.log(error)
    }
}
```

Cadastrando dados na API

Após isso, no arquivo **gameController.js**, criaremos a constante **createGame** que recebe uma função assíncrona. Nessa função será coletado os campos vindos do corpo da requisição POST, em seguida será chamado o método **Create()** do service para cadastrar os registros no banco. Em seguida, o controller retornará o código de status **201 (Created)**. Além disso, também é feito o tratamento de erro, retornando o código de status **500 (Erro interno do servidor)**. Lembre-se também de **exportar createGame** no final do arquivo.

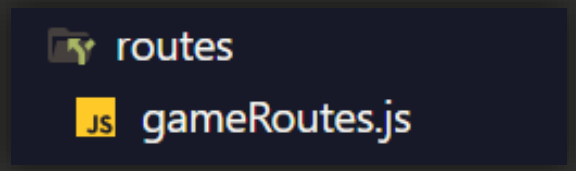
```
1  //Cadastrando um Game
2  const createGame = async (req, res) => {
3    try {
4      const { title, platform, year, price } = req.body;
5      await gameService.Create(title, platform, year, price);
6      res.sendStatus(201); //Código 201 (CREATED)
7    } catch (error) {
8      console.log(error);
9      res.status(500).json({ error: "Erro interno do servidor." });
10   }
```



Cadastrando dados na API

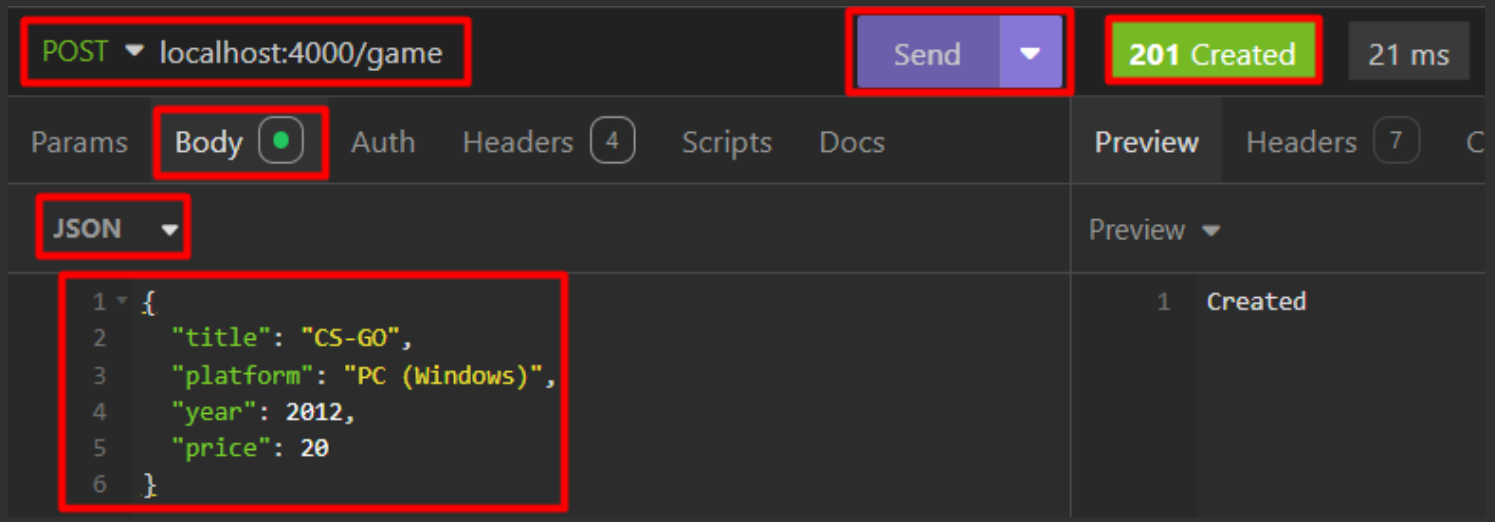
Agora devemos apenas criar um novo **endpoint** no arquivo **gameRoutes.js**, que receberá a requisição **POST** e chamará o método **createGame()** do gameController para tratar a requisição:

```
1  // Endpoint para cadastrar um Game
2  gameRoutes.post("/game", gameController.createGame);
```



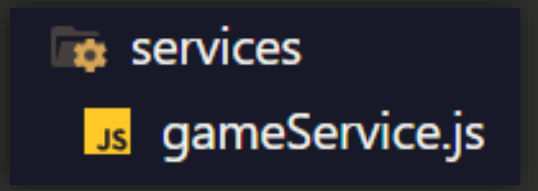
Feito isso, testaremos a requisição no **Insomnia**, enviando uma requisição **POST** para a rota "**/game**".

Será necessário enviar os dados que se deseja cadastrar no corpo da requisição (**BODY**) em formato **JSON**.



Deletando dados na API

Para deletar dados na API, criaremos um novo método chamado **Delete()** na classe **gameService** que irá excluir os registros no banco de dados.

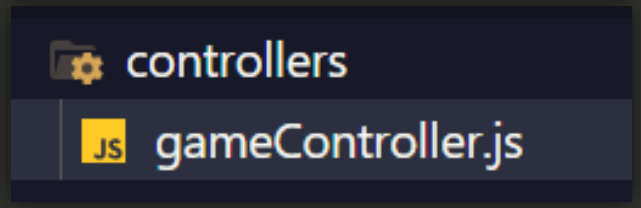



```
1  async Delete(id) {
2      try {
3          await Game.findByIdAndDelete(id);
4          console.log(`Game com a id: ${id} foi deletado.`)
5      } catch (error) {
6          console.log(error)
7      }
8  }
```

Deletando dados na API

Após isso, no arquivo **gameController.js**, criaremos a constante **deleteGame** que recebe uma função assíncrona. Nessa função será verificado se o ID do jogo enviado é válido. Para isso, precisamos instalar e importar o **ObjectId** da biblioteca **mongodb**. Após essa verificação, será chamado o método **Delete()** do service para excluir o registro em questão do banco de dados. Em seguida, o controller retornará o código de status **204 (No Content)**. Além disso, também é feito o tratamento de erro, retornando o código de status **400 (Bad Request)** ou **500**. Lembre-se também de **exportar deleteGame** no final.

```
1  //Deletando um Game
2  const deleteGame = async (req, res) => {
3      try {
4          if(ObjectId.isValid(req.params.id)){
5              const id = req.params.id
6              gameService.Delete(id)
7              res.sendStatus(204) // Código 204 (NO CONTENT) :
8          }else{
9              res.sendStatus(400) // Código 400 (BAD REQUEST) :
10         }
11     } catch (error) {
12         console.log(error)
13         res.status(500).json({ error: 'Erro interno do servidor.'})
14     }
15 }
```



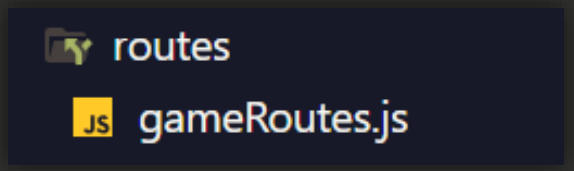

```
npm install mongodb
```

```
1  import {ObjectId} from 'mongodb'
```

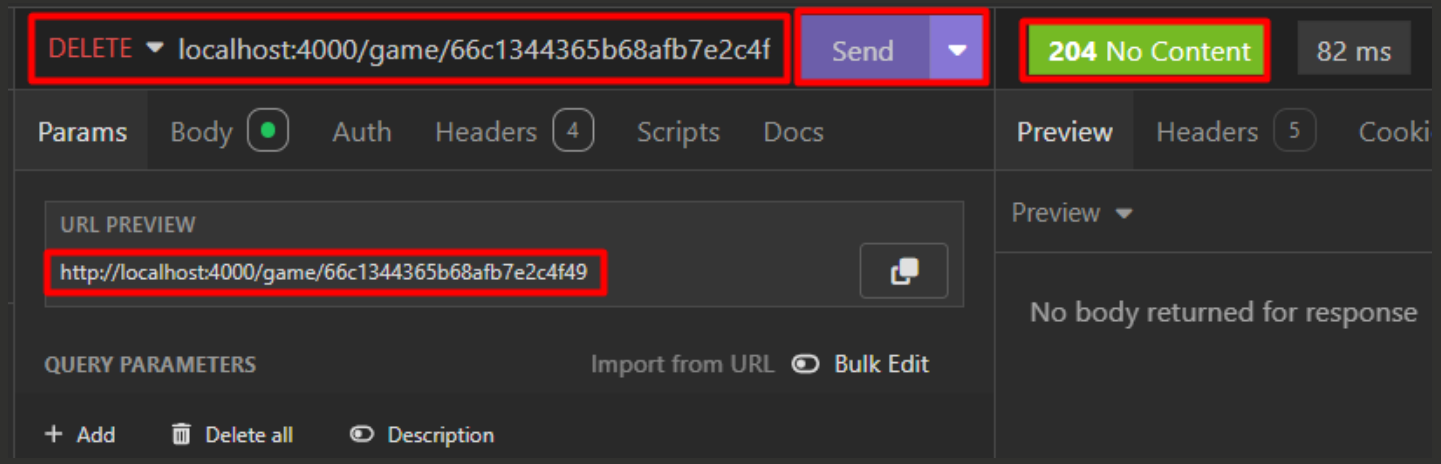
Deletando dados na API

Agora devemos apenas criar um novo **endpoint** no arquivo **gameRoutes.js**, que receberá a requisição **DELETE** e chamará o método **deleteGame()** do gameController para tratar a requisição. Essa rota terá um **parâmetro obrigatório** que será a **ID** do jogo que se deseja excluir:

```
1  // Endpoint para deletar um Game
2  gameRoutes.delete("/game/:id", gameController.deleteGame);
```

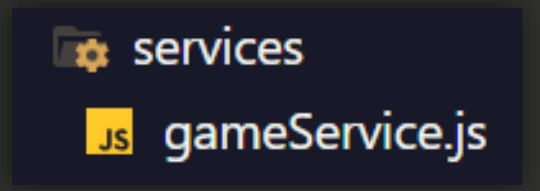


Feito isso, testaremos a requisição no **Insomnia**, enviando uma requisição **DELETE** para a rota "**/game**". Lembre-se que será necessário enviar a ID do jogo que se deseja excluir na URL:



Alterando dados na API

Para começarmos a alterar dados na API, criaremos um novo método chamado **Update()** na classe **gameService** que irá alterar registros no banco de dados.

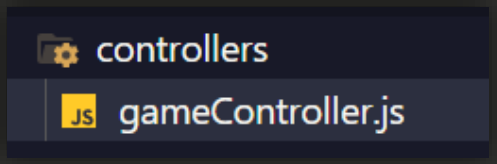



```
1  async Update(id, title, platform, year, price) {
2    try {
3      const updatedGame = await Game.findByIdAndUpdate(
4        id,
5        {
6          title,
7          platform,
8          year,
9          price,
10       },
11       { new: true }
12     );
13     console.log(`Dados do game com id: ${id} alterados com sucesso.`);
14     return updatedGame;
15   } catch (error) {
16     console.log(error);
17   }
18 }
```

Alterando dados na API

Após isso, no arquivo **gameController.js**, criaremos a constante **updateGame** que recebe uma função assíncrona. Nessa função será coletado os campos vindos do corpo da requisição PUT, em seguida será chamado o método **Update()** do service para alterar o registro no banco. O controller retornará o código de status **200 (OK)** e um **JSON** com os dados do jogo alterado. Lembre-se também de **exportar updateGame**.

```
1  //Alterando um Game
2  const updateGame = async (req, res) => {
3    try {
4      if (ObjectId.isValid(req.params.id)) {
5        const id = req.params.id;
6        const { title, platform, year, price } = req.body;
7        const game = await gameService.Update(id, title, platform, year, price);
8        res.status(200).json({ game }); //Código 200 (OK)
9      } else {
10       res.sendStatus(400); //Código 400 (BAD REQUEST)
11     }
12   } catch (error) {
13     console.log(error);
14     res.sendStatus(500); // Erro interno do servidor
15   }
16 };
```



Alterando dados na API

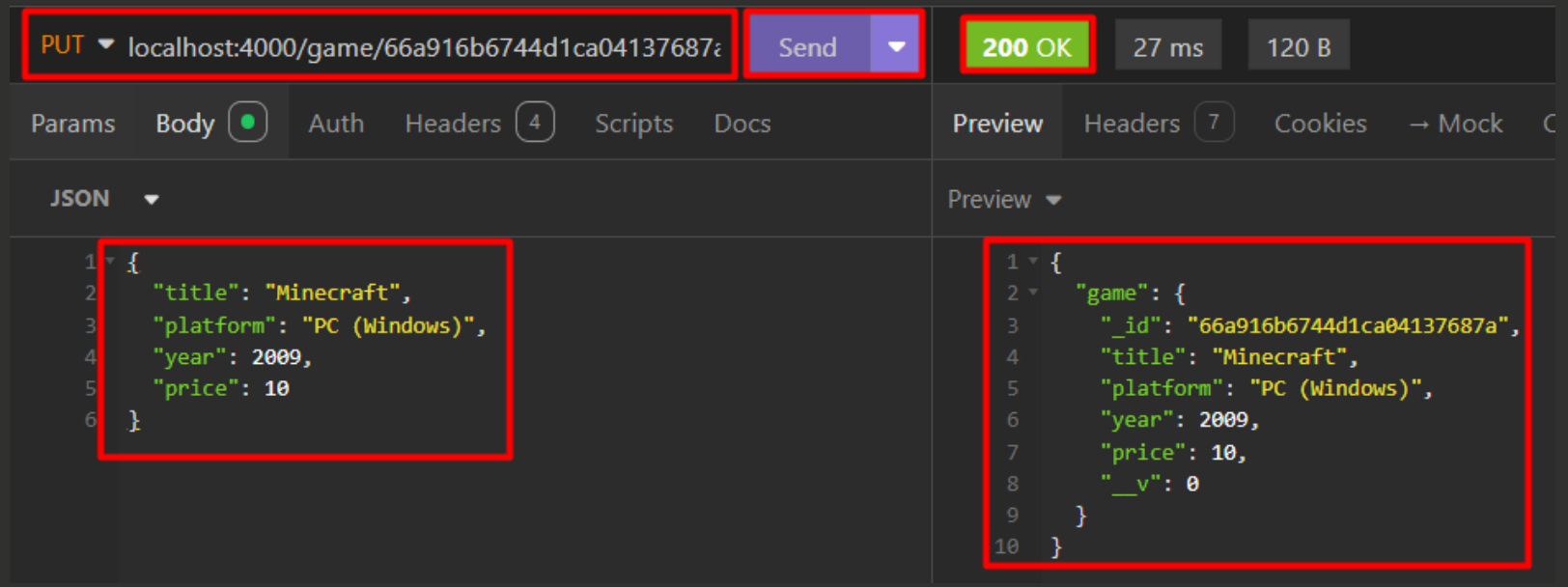
Agora devemos apenas criar um novo **endpoint** no arquivo **gameRoutes.js**, que receberá a requisição **PUT** e chamará o método **updateGame()** do gameController para tratar a requisição:

```
1  // Endpoint para alterar um Game
2  gameRoutes.put("/game/:id", gameController.updateGame);
```

routes

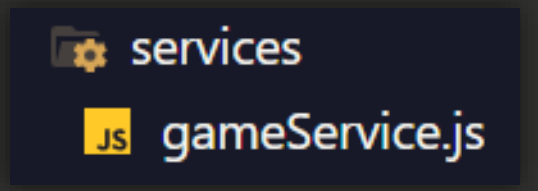
gameRoutes.js

Feito isso, testaremos a requisição no **Insomnia**, enviando uma requisição **PUT** para a rota "**/game**".
Será necessário enviar os dados que se deseja alterar no corpo da requisição (**BODY**) em formato **JSON**, bem como a **ID do game** na URL.



Listando um registro único

Para listar um registro da API, criaremos um novo método chamado **getOne()** na classe **gameService** que irá listar um único registro do banco de dados.

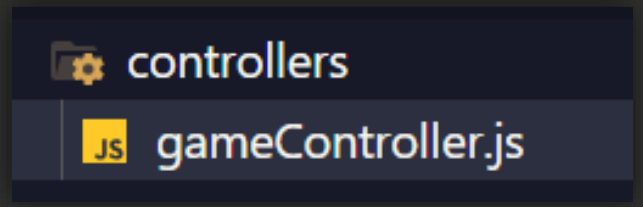



```
async getOne(id) {
    try {
        const game = await Game.findOne({_id: id})
        return game
    } catch (error) {
        console.log(error)
    }
}
```

Listando um registro único

Após isso, no arquivo **gameController.js**, criaremos a constante **getOneGame** que recebe uma função assíncrona. Nessa função será verificado se o ID do jogo enviado é válido. Após essa verificação, será chamado o método **getOne()** do service para buscar o registro em questão no banco de dados.
Em seguida, o controller retornará o código de status **200 (OK)**. E um **JSON** com os dados do jogo.

```
1 //Listando um único Game
2 const getOneGame = async (req, res) => {
3     try {
4         if (ObjectId.isValid(req.params.id)) {
5             const id = req.params.id
6             const game = await gameService.getOne(id)
7             if (!game) {
8                 res.sendStatus(404) // Jogo não encontrado
9             } else {
10                res.status(200).json({ game })
11            }
12        } else {
13            res.sendStatus(400) // Requisição inválida - Bad request
14        }
15    } catch (error) {
16        console.log(error)
17        res.sendStatus(500) // Erro interno do servidor
18    }
19}
20
21export default { getAllGames, createGame, deleteGame, updateGame, getOneGame }
```



Listando um registro único

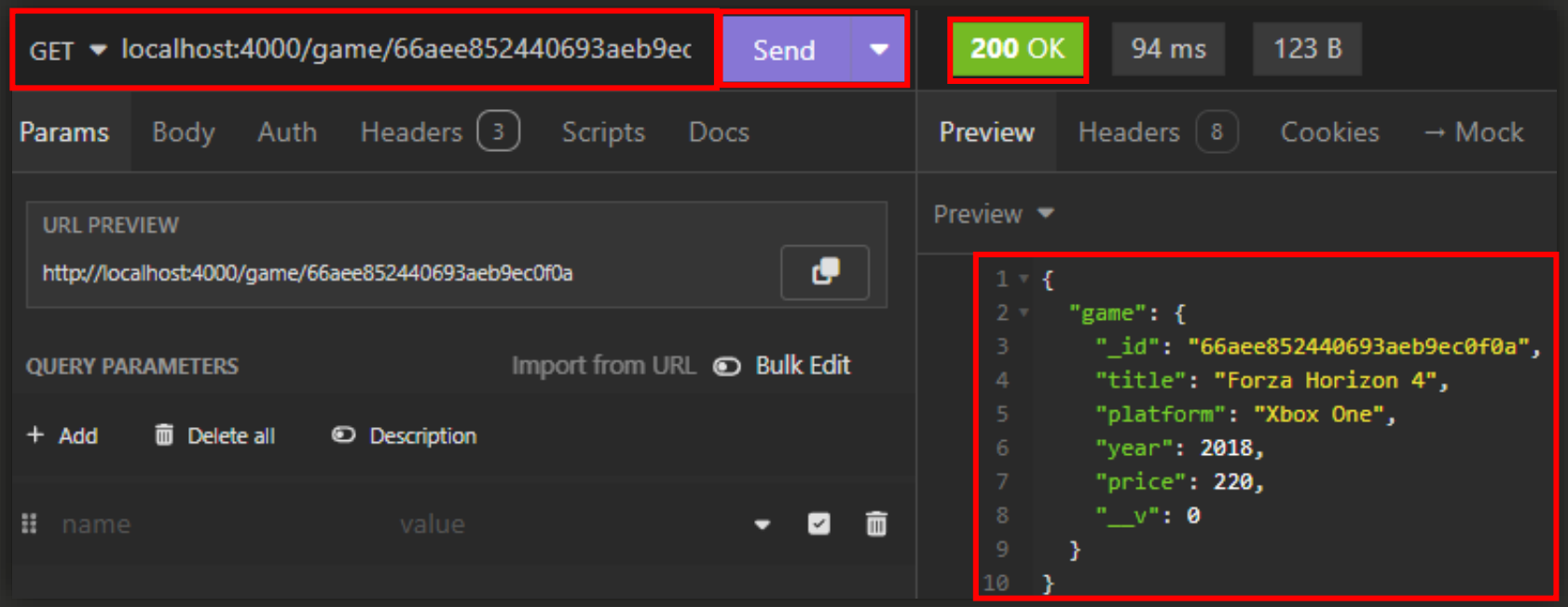
Agora devemos apenas criar um novo **endpoint** no arquivo **gameRoutes.js**, que receberá a requisição **GET** e chamará o método **getOneGame()** do gameController para tratar a requisição. Essa rota terá um **parâmetro obrigatório** que será a **ID** do jogo que se deseja buscar:

```
1 // Endpoint para listar um único Game
2 gameRoutes.get("/game/:id", gameController.getOneGame)
```

routes

gameRoutes.js

Feito isso, testaremos a requisição no **Insomnia**, enviando uma requisição **GET** para a rota "**/game**". Lembre-se que será necessário enviar a ID do jogo que se deseja buscar na URL:

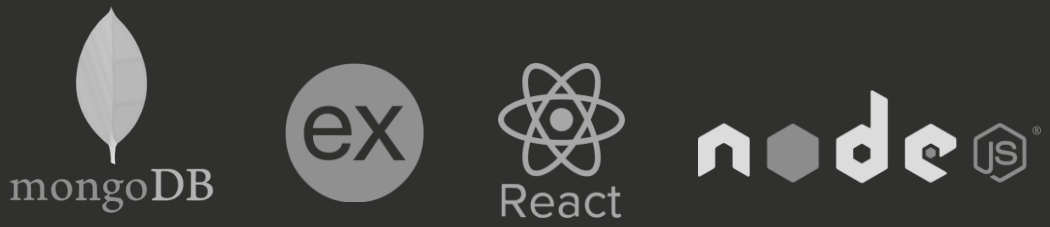


Exercício de fixação – Aula 01 e 02:

https://forms.cloud.microsoft/r/m1rrvZj2fH









Aula 02:

Desenvolvendo o CRUD da API